

高级语言程序设计

实验报告

南开大学 计算机大类

刘哲维

2511639

软件 4 班

2026/5/15

一、、 作业题目

基于 Qt C++ 的三国杀 1v1 卡牌对战游戏——势魏延 VS 界徐盛。

本课题来源于对三国杀桌游的计算机化实现。我选择势魏延和界徐盛这两个武将作为对战双方，因为这两个角色的技能体系完整且相互制衡：势魏延拥有壮誓、饮战、狂骨、忠傲（使命技）、背水、困奋六个技能，界徐盛则拥有标志性的破军技能。二者对战的策略深度足够展现程序设计的复杂性。

二、、 开发软件

- Qt Creator 6.x (Qt 6.2.4) ——图形界面开发框架
- C++17 标准——使用 MSVC 2019 编译器
- Git 2.x ——版本控制 (GitHub 托管)
- Visual Studio Code ——辅助代码编辑
- L^AT_EX ——实验报告排版

三、、 课题要求

1. 使用 C++ 语言完成图形化小程序，不得使用其他编程语言实现核心逻辑
2. 面向对象设计：合理使用类、继承、多态等机制组织代码
3. 创新创意主题：选题需具有原创性，不可提交已有开源项目的直接复制品
4. 完整的项目迭代提交：使用 Git 管理版本，提交记录需体现迭代过程（至少 3 次提交）
5. 项目讲解视频：录制视频上传至 B 站，展示游戏运行效果并讲解技术要点

四、 主要流程

1.、 整体流程

(1)、 设计思路

本项目的核心设计遵循面向对象的基本原则，将游戏中的实体和逻辑抽象为若干 C++ 类，通过类的继承体系实现代码复用，通过虚函数多态实现卡牌效果的灵活扩展。

(2)、 核心类设计

(1) Player 玩家基类

Player 类是所有武将的基类，封装了玩家的基本属性和操作。类的私有成员通过公有方法对外提供受控访问，这是封装特性的直接体现。

```
1 struct EquipmentArea {
2     std::shared_ptr<Card> weapon;
3     std::shared_ptr<Card> armor;
4     std::shared_ptr<Card> plusHorse;
5     std::shared_ptr<Card> minusHorse;
6 };
7
8 class Player {
9 public:
10     QString name;           // 玩家名称
11     int hp;                  // 当前体力值
12     int maxHp;               // 最大体力值
13     std::vector<std::shared_ptr<Card>> handCards; // 手牌列表
14     EquipmentArea equipment; // 装备区域
15     std::vector<std::shared_ptr<Card>> kouZhiCards; // 被破军扣
    置的牌
16     int slashUsedThisTurn = 0; // 本回合已使用杀的数量
    量
17     int slashCountLimit = 1; // 本回合杀的使用上限
```

```
18
19     Player(QString n, int h) : name(n), hp(h), maxHp(h) {}
20     virtual ~Player() {}
21
22     void takeDamage(int amount);           // 受到伤害（含白银狮子减伤逻辑）
23     void heal(int amount);                 // 回复体力（不超过上限）
24     void loseHp(int amount);               // 失去体力（区别于伤害）
25     void drawCard(std::shared_ptr<Card> card);
26     void discardRandom(int count);
27     bool isAlive() const { return hp > 0; }
28     void equipCard(std::shared_ptr<Card> card);
29
30     int getCurrentDistanceTo(Player* target) const;
31     int getAttackRange() const;
32     bool hasSlash() const;
33     bool playSlash();
34     // ... 装备检查方法
35 };
```

Listing 1: Player.h ——玩家基类的核心结构

设计要点:

- hp 和 maxHp 为 public 成员，但修改血量的操作（伤害、治疗、失去体力）必须通过 takeDamage()、heal()、loseHp() 方法完成，这些方法内部包含了边界检查和装备效果处理（如白银狮子减伤），外部代码无法绕过这些逻辑直接修改血量。
- takeDamage() 内部检查白银狮子装备效果，超过 1 点的伤害自动减为 1 点，体现了封装带来的业务逻辑内聚。
- 析构函数声明为 virtual，确保通过基类指针删除派生类对象时能正确调用派生类的析构函数，这是 C++ 多态机制的基础要求。

(2) WeiYan 和 XuSheng 派生类

WeiYan 和 XuSheng 均继承自 Player，在复用基类属性和方法的基础上，各自实现专属技能。

```

1  class WeiYan : public Player {
2  public:
3      WeiYan() : Player("势·魏延", 5), zhongAoState(
4          MISSION_ACTIVE),
5          kuangGuLevel(1), zhuangShiDiscarded(0), zhuangShiHpLost
6              (0),
7          zhuangShiUsedThisTurn(false), cardsUsedThisTurn(0),
8          zhuangShiNoLimitRemaining(0), zhuangShiNoLimitUsed(0),
9          zhuangShiNoCountRemaining(0), zhuangShiNoCountUsed(0),
10         slashUsedThisTurn(0), slashCountLimit(1),
11         chengShiCanTrigger(false), chengShiDiscardedCard(
12             nullptr) {}
13
14     // 忠傲使命状态
15     ZhongAoState zhongAoState;
16     int kuangGuLevel;
17
18     // 壮誓技能状态
19     int zhuangShiDiscarded, zhuangShiHpLost,
20         zhuangShiNoLimitRemaining, zhuangShiNoCountRemaining;
21     // ...
22
23     void activateZhuangShi(int discard, int hpLoss); // 壮誓
24     void skipZhuangShi(); // 放弃
25         壮誓
26
27     std::pair<bool, int> useSlash(Player* target, // 饮战
28         int baseDamage = 1);
29
30     void chooseKuangGuHeal(); // 狂骨-
31         回血
32
33     void chooseKuangGuDraw(); // 狂骨-
34         摸牌

```

```

26     void activateBeiShui();           // 背水
27     void onKill(Player* target);      // 忠傲
        成功
28     void onDamageDealt(Player* target, int damage); // 造成
        伤害
29     void endTurn();                   // 回合
        结束
30     void startTurn();                 // 回合
        开始
31 };

```

Listing 2: WeiYan.h —— 势魏延继承 Player

基类		派生类	
Player		WeiYan	XuSheng
name, hp, maxHp	共用属性	继承	继承
takeDamage()	共用方法	继承	继承
heal()	共用方法	继承	继承
—	新增技能	壮誓、饮战、狂骨、忠傲、背水、困奋	破军
—	体力	5	4

图 1: 继承关系与属性分布

(3) Card 卡牌类体系

卡牌系统是多态设计的核心。Card 作为抽象基类，定义了纯虚函数 execute()，所有具体卡牌必须重写此方法来实现各自的效果。

```

1  enum class CardCategory { BASIC, TRICK, EQUIPMENT, OTHER };
2
3  class Card {
4  public:
5      CardCategory category;
6      QString name;
7

```

```

8      Card(CardCategory cat, QString n) : category(cat), name(n)
9          {}
10
11     virtual ~Card() {}
12
13     // 纯虚函数：每个具体卡牌必须实现自己的 execute()
14     virtual void execute(Player* source, Player* target) = 0;
15
16     // AOE锦囊对多目标执行
17     virtual void executeAOE(Player* source,
18                             const std::vector<Player*>& targets
19                             ) {
20         for (auto target : targets) {
21             execute(source, target);
22         }
23     };

```

Listing 3: Card.h ——卡牌基类与纯虚函数

类继承层次:

Card 为抽象基类，定义纯虚函数 `execute()`，所有卡牌均继承自它。其下派生出三个分支：

Card (抽象基类)

- BasicCard——基本牌：SlashCard (杀)、DodgeCard (闪)、PeachCard (桃)、WineCard (酒)
- TrickCard——锦囊牌：DuelCard (决斗)、NanmanCard (南蛮入侵)、ArrowCard (万箭齐发)、StealCard (顺手牵羊)、BridgeCard (过河拆桥)
- EquipCard——装备牌：WeaponCard (武器)、ArmorCard (防具)、HorseCard (坐骑)

多态的使用：在游戏主循环中，所有卡牌以 `std::shared_ptr<Card>` 类型统一存储于手牌列表和牌堆中。调用卡牌效果时，直接通过基类指针调用 `card->execute(source, target)`，C++ 运行时会根据卡牌的实际类型自动分发到

正确的 `execute()` 实现。这使得添加新卡牌类型只需新增一个派生类，无需修改已有代码。

(3)、 状态机游戏流程

项目使用 `enum class FlowState` 枚举来管理游戏的各个阶段。每个回合从摸牌到出牌到结束，由一个状态变量控制流转。MainWindow 中的 `updateUI()` 方法根据当前状态更新界面。

```
1 enum class FlowState {
2     START_MENU,           // 开始菜单
3     TURN_DRAW,            // 摸牌阶段
4     TURN_PLAY_SELECT,     // 出牌阶段（选择牌）
5     TURN_PLAY_CONFIRM,    // 出牌阶段（确认）
6     WAIT_RESPONSE,       // 等待响应
7     RESOLVE,              // 结算阶段
8     TURN_END,             // 回合结束
9     DISCARD_PHASE,       // 弃牌阶段
10    ZHUANGSHI_SELECT,     // 壮誓选择阶段
11    ZHUANGSHI_DISCARD,    // 壮誓弃牌阶段
12    NEAR_DEATH,           // 濒死阶段
13    POJUN_SELECT,         // 破军选牌阶段
14    POJUN_DODGE_WAIT,     // 破军等待出闪
15    KUNFEN_END_PHASE,     // 困奋结束阶段
16    // ...
17 };
```

Listing 4: mainwindow.h ——状态机枚举

状态流转说明：

游戏每个回合严格按照以下顺序流转：

开始菜单 → 摸牌阶段 → 出牌阶段 → 结算/等待响应 → 回合结束 → 弃牌阶段 → 下一回合摸牌阶段
--

各阶段中插入的特殊子状态对照如下：

- **出牌阶段**：可插入 **壮誓选择**（势魏延专属）
- **结算阶段**：可插入 **破军选牌**（界徐盛专属）
- **回合结束前**：检测 **濒死判定**
- **弃牌阶段后**：检测 **困奋阶段**（忠傲失败时触发）

所有状态切换统一由 `enterFlowState()` 方法管理，保证状态流转的一致性和可追溯性。

2.、 算法或公式

(1)、 壮誓技能算法

触发条件：出牌阶段，忠傲使命未失败，魏延选择发动壮誓。

输入：弃牌数 N ，掉血数 M 。

效果：

- 弃置 N 张手牌 → 接下来 N 张使用的牌无距离限制且不可被响应
- 失去 M 点体力 → 接下来 M 张使用的牌不计入使用次数

```

1 void WeiYan::activateZhuangShi(int discard, int hpLoss) {
2     if (zhongAoState == MISSION_FAILED) return;
3     zhuangShiUsedThisTurn = true;
4
5     if (discard > 0) {
6         zhuangShiDiscarded = discard;
7         zhuangShiNoLimitRemaining = discard; // 无距离限制计数器
8     }
9     if (hpLoss > 0) {
10        zhuangShiHpLost = hpLoss;
11        zhuangShiNoCountRemaining = hpLoss; // 不计入使用次数计数器
12    }
    this->loseHp(hpLoss);

```

```
13     }
14 }
```

Listing 5: WeiYan.cpp ——activateZhuangShi 实现

算法核心：用两个计数器 (zhuangShiNoLimitRemaining 和 zhuangShiNoCountRemaining) 追踪剩余次数。每次使用一张牌后计数器减 1，为 0 时效果失效。距离计算时优先检查计数器状态：

```
1 int WeiYan::getCurrentDistanceTo(Player* target) const {
2     if (zhuangShiNoLimitRemaining > 0) {
3         return 1; // 壮誓效果：无视距离
4     }
5     // 正常距离计算
6     int distance = 1;
7     if (this->equipment.minusHorse) distance--;
8     if (target->equipment.plusHorse) distance++;
9     if (distance < 1) distance = 1;
10    return distance;
11 }
```

Listing 6: WeiYan.cpp ——距离计算重写

相关 C++ 知识点：函数重写 (override)、条件分支控制、成员状态管理。

(2)、 饮战技能算法

触发条件：势魏延使用【杀】指定目标时。

判断逻辑：

1. **条件一：**若魏延当前体力 $\leq$ 目标体力 $\rightarrow$ 此杀伤害 +1
2. **条件二：**若魏延手牌数 $\leq$ 目标手牌数 $\rightarrow$ 结算后弃置目标一张牌
3. **乘势：**若条件一和条件二同时满足 $\rightarrow$ 触发乘势

```
1 std::pair<bool, int> WeiYan::useSlash(Player* target, int
    baseDamage) {
```

```
2     int damage = baseDamage;
3     bool yingZhanJiaShang = false;
4     bool yingZhanQiPai = false;
5
6     // 条件1: 体力    目标 → 伤害+1
7     if (this->hp <= target->hp) {
8         damage += 1;
9         yingZhanJiaShang = true;
10    }
11    // 条件2: 手牌    目标 → 弃目标一张牌
12    if ((int)this->handCards.size() <= (int)target->handCards.
13        size()) {
14        yingZhanQiPai = true;
15    }
16    // 乘势: 两个条件都满足
17    if (yingZhanJiaShang && yingZhanQiPai) {
18        qDebug() << "【乘势】两个条件均满足! ";
19    }
20
21    target->takeDamage(damage);
22    slashUsedThisTurn++;
23    return std::make_pair(yingZhanQiPai, damage);
24 }
```

Listing 7: WeiYan.cpp ——useSlash 饮战实现

算法核心：两个独立的布尔条件判断，通过 `std::pair<bool, int>` 同时返回”是否需要弃牌”和”实际伤害值”两个结果。使用 `std::make_pair()` 构造返回值，调用方通过 `.first` 和 `.second` 分别获取。

相关 C++ 知识点： `std::pair` 的使用、条件分支、`static_cast<int>` 类型转换。

(3)、破军扣牌与闪判断算法

触发条件：界徐盛使用【杀】造成伤害后，目标存活。

效果：可扣置目标最多 X 张手牌和装备区的牌，其中 X = 目标当前体力值。

```
1 void XuSheng::useSlash(Player* target, int baseDamage) {
2     if (slashUsedThisTurn >= slashCountLimit) return;
3
4     slashUsedThisTurn++;
5     target->takeDamage(baseDamage);
6
7     // 破军触发条件：造成伤害且目标存活
8     if (baseDamage > 0 && target->isAlive()) {
9         pojunTarget = target;
10        pojunKouZhiCount = target->hp; // 可扣牌数 = 目标当前
11        体力
12    }
13 }
```

Listing 8: XuSheng.cpp ——useSlash 破军触发

破军扣牌后的闪判断流程：

1. 徐盛从目标的 handCards 和 equipment 中选择最多 pojunKouZhiCount 张牌扣置
2. 扣置的牌移入目标的 kouZhiCards 列表（目标暂时失去这些牌）
3. 判定目标剩余手牌中是否有【闪】：
 - 有闪 → 自动出闪，避免伤害，扣置牌归还
 - 无闪 → 伤害结算，扣置牌留在扣置区

相关 C++ 知识点： std::vector 的元素增删操作（push_back、erase）、指针传递、对象生命周期管理。

(4)、 狂骨/忠傲使命技算法

狂骨（一级）：魏延造成伤害后触发，每造成 1 点伤害可发动一次。选项：

1. 回复 1 点体力（若满血则改为摸 1 张牌）

2. 摸 1 张牌

3. 不发动

忠傲使命技 (状态机):

忠傲是一个”使命技”，包含三种状态：MISSION_ACTIVE、MISSION_SUCCESS、MISSION_FAILED。

```
1 enum ZhongAoState { MISSION_ACTIVE, MISSION_SUCCESS,
    MISSION_FAILED };
```

Listing 9: WeiYan.h ——忠傲状态枚举

状态转换:

忠傲使命技包含三种状态，转换规则如下:

三种状态:

- MISSION_ACTIVE (使命进行中) ——初始状态，可使用壮誓
- MISSION_SUCCESS (使命成功) ——击杀徐盛后进入
- MISSION_FAILED (使命失败) ——放弃壮誓后进入

转换路径与效果:

MISSION_ACTIVE $\xrightarrow{\text{击杀徐盛}}$ MISSION_SUCCESS

→ **成功奖励:** 狂骨升级为 2 级、解锁背水技能; 使用牌数 < 弃牌数
则摸 1 张; 使用牌数 < 掉血数则回复 1 点或摸 1 张

MISSION_ACTIVE $\xrightarrow{\text{放弃壮誓}}$ MISSION_FAILED

→ **失败惩罚:** 失去壮誓技能 获得困倦

```
1 void WeiYan::onKill(Player* target) {
2     if (zhongAoState != MISSION_ACTIVE) return;
3
4     kuangGuLevel = 2;                // 狂骨升级
5     zhongAoState = MISSION_SUCCESS;  // 使命成功
6
7     // 奖励1: 使用牌数 < 弃牌数 → 摸1张
8     if (cardsUsedThisTurn < zhuangShiDiscarded) {
```

```

9         if (gDeck && !gDeck->deck.empty())
10             this->drawCard(gDeck->draw());
11     }
12     // 奖励2: 使用牌数 < 掉血数 → 回复1点或摸1张
13     if (cardsUsedThisTurn < zhuangShiHpLost) {
14         if (this->hp < this->maxHp) this->heal(1);
15         else if (gDeck && !gDeck->deck.empty())
16             this->drawCard(gDeck->draw());
17     }
18 }

```

Listing 10: WeiYan.cpp ——onKill 忠傲成功

相关 C++ 知识点：枚举类型管理状态、条件判断、状态机模式。

(5)、 濒死自救贪心算法

触发条件：任一玩家体力降至 0 或以下时。

算法策略：贪心算法——按优先级依次使用自救卡牌。

```

1 void MainWindow::handleNearDeath(Player* player) {
2     // 贪心策略：优先用酒（更珍贵），再用桃
3     // 第一步：找酒
4     for (auto it = player->handCards.begin();
5         it != player->handCards.end(); ) {
6         if ((*it)->name == "酒") {
7             player->heal(1);
8             player->handCards.erase(it);
9             return;
10        } else { ++it; }
11    }
12    // 第二步：找桃
13    for (auto it = player->handCards.begin();
14        it != player->handCards.end(); ) {
15        if ((*it)->name == "桃") {
16            player->heal(1);

```

```
17         player->handCards.erase(it);
18         return;
19     } else { ++it; }
20 }
21 // 都没有 → 游戏结束
22 showGameOver(player);
23 }
```

Listing 11: mainwindow.cpp ——handleNearDeath 濒死自救

为什么这是贪心算法：每次濒死时，算法优先消耗价值更高的【酒】（酒在非濒死时可用于增伤，而桃只能回血），因此先搜索酒、再搜索桃的策略在当前状态下是局部最优选择。同时每次只使用一张牌（一旦找到就 return），避免浪费资源。

时间复杂度： $O(n)$ ，其中 n 为手牌数量。算法线性遍历手牌，最坏情况下需遍历两遍（先找酒后找桃）。

相关 C++ 知识点：迭代器遍历与删除（`vector::erase` 配合迭代器）、范围 for 循环等价写法、提前返回模式（early return）。

3.、 单元测试

(1)、 单元测试的设计思路与目标

单元测试是保证软件质量的重要手段。本项目中，我对每个核心技能编写了独立的测试用例，采用”输入–预期输出–实际输出”的三段式验证方法。

测试目标：

1. **技能逻辑正确性：**每个技能在给定输入下，输出是否符合规则设计
2. **边界条件：**血量下限（0 点即死亡）、血量上限（不能超过 maxHp）、空手牌时的行为
3. **装备交互：**白银狮子减伤、古锭刀增伤、诸葛连弩杀次数等装备与技能的联动
4. **状态流转：**状态机在各阶段的切换是否正确，无死循环或状态遗漏

测试框架：本项目规模适中，采用手动构造测试场景 + Qt 的 qDebug() 输出结果的方式进行测试。具体做法是：在代码中预设玩家状态和手牌，调用技能方法，通过 qDebug() 输出关键变量值，与预期进行人工比对。

五、、 单元测试

以下是 5 个核心测试用例的详细执行过程和结果。

(1)、 测试用例 1：壮誓技能

项目	内容
测试目标	验证壮誓技能计数器的正确性
输入	选择弃 2 张牌，掉 1 血，随后使用一张【杀】
预期输出	杀无距离限制、不可被响应、不计入使用次数； zhuangShiNoLimitRemaining 从 2 变为 1
实际结果	通过。计数器正确递减，杀可攻击任意距离目标

(2)、 测试用例 2：饮战技能

项目	内容
测试目标	验证饮战两个条件的判定及乘势触发
输入	魏延体力 4，徐盛体力 5；魏延手牌 3 张，徐盛手牌 4 张；魏延使用【杀】
预期输出	伤害 +1 ($4 \leq 5$)、需弃徐盛一张牌 ($3 \leq 4$)、触发乘势
实际结果	通过。useSlash 返回 pair(true, 2)，乘势标记正确设置

(3)、 测试用例 3：破军技能

项目	内容
测试目标	验证破军扣牌及闪判断逻辑
输入	徐盛造成伤害，目标体力 2，扣置 2 张牌
预期输出	目标有闪则自动出闪避免伤害，无闪则结算伤害
实际结果	通过。两种分支均测试通过，闪判断逻辑正确

(4)、 测试用例 4：濒死自救

项目	内容
测试目标	验证濒死自救的贪心策略（先酒后桃）
输入	玩家濒死，手牌中有 1 张【酒】和 1 张【桃】
预期输出	优先使用【酒】自救，【桃】保留在手牌中
实际结果	通过。按照预期顺序，先消耗酒再消耗桃

(5)、 测试用例 5：造成 3 点伤害音效

项目	内容
测试目标	验证多音效播放时不互相覆盖
输入	酒 + 破军 + 饮战条件满足，造成 3 点伤害
预期输出	先播放饮战音效，延迟 1 秒后播放”癫狂屠戮”音效，两者不重叠覆盖
实际结果	通过。使用 QTimer::singleShot(1000, ...) 延迟播放第二个音效，音效顺序正确

六、、 收获

通过这个项目，我将课堂上学到的 C++ 理论知识应用到了实际开发中，以下是我在六个核心方面的具体收获。

1.、 封装与数据保护

在编写 Player 类时，我深刻理解了封装的意义。血量 hp 虽然是 public 成员，但所有的血量修改都必须经过 takeDamage()、heal()、loseHp() 这些方法。原因在于：

- takeDamage() 内部需要检查白银狮子的减伤效果
- heal() 需要确保血量不超过 maxHp
- loseHp() (失去体力) 与 takeDamage() (受到伤害) 是两种不同的概念，前者不触发装备减伤

如果外部代码直接写 `player.hp -= 3`，就会绕过这些保护逻辑，导致游戏出现 Bug。封装让数据修改必须经过统一的入口，保证了逻辑的一致性。

2.、 继承与代码复用

WeiYan 和 XuSheng 两个武将类都继承自 Player，这是我第一次在实践中感受到继承带来的“代码复用”究竟是什么。两个武将共用一套血量管理、手牌操作、装备系统、距离计算的代码，我只需要在各自的派生类中编写技能相关的逻辑即可。如果不使用继承，每个武将类都要重复写一遍 Player 的代码，不仅工作量大，修改时还需要两处同步更新。

3.、 多态与虚函数机制

Card 基类的 `virtual void execute() = 0` 纯虚函数是项目中最让我体会到多态价值的设计。不同卡牌的效果千差万别——杀造成伤害、桃回复体力、顺手牵羊夺取手牌——但在游戏主循环中，它们都通过统一的方式调用：

```
card->execute(source, target)。
```

我理解了虚函数表 (vtable) 的工作原理：每个含虚函数的类在编译时生成一张虚函数表，对象通过 vtable 指针找到正确的 execute() 实现。这使得添加新卡牌只需新建一个派生类并重写 execute()，完全不需要修改游戏的调度代码，遵循了“对扩展开放、对修改关闭”的原则。

另外, Player 的析构函数声明为 `virtual Player() {}` 也是必要的——通过基类指针 `Player*` 删除 `WeiYan` 或 `XuSheng` 对象时, 虚析构函数保证派生类的析构函数先被调用, 避免内存泄漏。

4.、 STL 容器与智能指针

项目中大量使用了 C++ 标准库提供的容器和智能指针:

- `std::vector<std::shared_ptr<Card>>` ——手牌和牌堆的管理。我一开始考虑过用裸指针 `Card*` 配合 `new/delete`, 但很快就遇到了“一张牌被多个地方引用时谁来 `delete`”的问题。改用 `shared_ptr` 后, 引用计数自动管理内存, 彻底消除了悬空指针和重复释放的隐患。
- `std::pair<bool, int>` ——`WeiYan::useSlash` 需要同时返回“是否需要弃牌”和“实际伤害值”两个结果。用 `pair` 比定义一个新的结构体更轻量, 也比用两个引用参数传回结果更清晰。
- `auto` 关键字——对于 `std::vector<std::shared_ptr<Card>>::iterator` 这种冗长的类型名, `auto it = ...` 让代码简洁很多。
- `enum class` ——使用强类型枚举定义 `FlowState`, 相比传统 `enum` 不会隐式转换为 `int`, 避免了枚举值之间的意外比较。

5.、 洗牌算法 (Fisher-Yates)

牌堆的随机性是卡牌游戏公平性的基础。我直接使用了 C++11 标准库提供的 `std::shuffle` 函数, 它内部实现了 Fisher-Yates 洗牌算法。配合 `std::mt19937` (梅森旋转算法) 作为随机数引擎, 生成的牌堆顺序在统计上是均匀的——每一张牌出现在每一个位置的概率相等。

```
1 void DeckManager::shuffle() {  
2     std::random_device rd;  
3     std::mt19937 g(rd());  
4     std::shuffle(deck.begin(), deck.end(), g);  
5 }
```

Listing 12: DeckManager.cpp ——洗牌算法

我对比过 `std::random_shuffle` (C++14 已废弃) 和 `std::shuffle` 的区别：前者依赖 `rand()`，随机性差且有模偏差；后者必须显式传入随机数引擎，质量控制更好。这也是 C++ 新标准改进 API 设计的一个好例子。

6.、 状态机设计模式

游戏流程复杂，包含摸牌、出牌、结算、濒死、弃牌、壮誓选择、破军选择、困奋等多个阶段。我用一个 `FlowState` 枚举和 `switch-case` 实现了简单的状态机。每个状态对应一段独立的处理代码，状态的切换通过 `enterFlowState()` 方法统一管理。

这个设计让我体会到状态机模式的核心优势：每个状态的逻辑互不干扰，添加新状态时只需在枚举中增加一项并在 `switch` 中添加一个 `case`，不会影响已有状态的处理逻辑。Debug 时也可以通过当前状态快速定位问题所在的代码段。

七、、 AI 工具使用披露

AI 工具	用途	使用位置	说明
Trae AI / Claude	代码调试辅助	mainwindow.cpp、WeiYan.cpp 等	协助定位编译错误和逻辑 Bug
Trae AI / Claude	代码注释生成	所有.h/.cpp 文件	为代码添加中文注释，提高可读性
Trae AI / Claude	视频文案	视频脚本	B 站讲解视频的文案草稿
Trae AI / Claude	L ^A T _E X 排版	本报告	实验报告的 L ^A T _E X 排版美化

说明：项目的核心设计——包括类的继承体系、卡牌多态机制、状态机结构、各技能的算法逻辑——均由我独立设计和实现。AI 工具主要辅助了代码调试、注释生成和文档排版等工作，项目的整体架构和关键代码的编写由我本人完成。